

|                 |                                                                           |
|-----------------|---------------------------------------------------------------------------|
| Document number | P0746R0                                                                   |
| Date            | 2017-07-13                                                                |
| Project         | Programming Language C++, Library Working Group                           |
| Reply-to        | Jonathan Wakely <cxx@kayari.org>, Michael McLaughlin <mikebmcl@gmail.com> |

---

# Wording for Networking PDTS ballot comment 011 (US-10)

## Introduction

This paper presents proposed changes in response to one of the National Body comments for PDTS 19216 from [N4643](#).

## Changes

### NB comment 011 (US-10)

Without knowing more about what kind of executor is being used, a user will have difficulty deciding which of the three functions to use to add a task to an executor. It is preferable to limit the options for adding tasks to a generic executor. Concrete executors can add additional functions if required.

**Proposed change:** Remove the `defer` function from executors, as that is the least well-defined. This would match the existing Boost ASIO implementation.

This was discussed by LEWG in Kona and rejected, then discussed again in Toronto due to the [P0703R0](#) paper. LEWG preferred to keep the `defer` function but add normative recommendation to distinguish it from `post`. The wording below was provided by Michael McLaughlin.

The change is to split the row for `post` and `defer` into two rows, with differing normative wording. The revised wording for `post` says "The executor may begin `f1`'s progress before the call to `post` completes." The revised wording for `defer` says "The executor should not begin `f1`'s progress before the call to `defer` completes."

## Proposed Wording

Changes are relative to [N4656](#).

Modify 13.2.2 [async.reqmts.executor] Table 4 as shown:

### expression

```
x1.post(std::move(f), a)
x1.defer(std::move(f), a)
```

### type

[Drafting note: empty cell]

## assertion/note pre/post-condition

*Effects:* Creates an object `f1` initialized with `DECAY_COPY(forward<Func>(f))` in the current thread of execution. Calls `f1()` at most once. The executor shall not block forward progress of the caller pending completion of `f1()`. The executor may begin `f1`'s progress before the call to `post` completes. Executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. --end note]

*Synchronization:* The invocation of `post` ~~or `defer`~~ synchronizes with (C++Std [intro.multithread]) the invocation of `f1`. ~~[Note: Although the requirements placed on `defer` are identical to `post`, the use of `post` conveys a preference that the caller does not block the first step of `f1`'s progress, whereas `defer` conveys a preference that the caller does block the first step of `f1`. One use of `defer` is to convey the intention of the caller that `f1` is a continuation of the current call context. The executor may use this information to optimize or otherwise adjust the way in which `f1` is invoked. --end note]~~

## expression

`x1.defer(std::move(f),_a).`

## type

[Drafting note: empty cell]

## assertion/note pre/post-condition

*Effects:* Creates an object `f1` initialized with `DECAY_COPY(forward<Func>(f))` in the current thread of execution. Calls `f1()` at most once. The executor shall not block forward progress of the caller pending completion of `f1()`. The executor should not begin `f1`'s progress before the call to `defer` completes. [Note: One use of `defer` is to convey the intention of the caller that `f1` is a continuation of the current call context. The executor may use this information to optimize or otherwise adjust the way in which `f1` is invoked. --end note] Executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. --end note]

*Synchronization:* The invocation of `defer` synchronizes with (C++Std [intro.multithread]) the invocation of `f1`.